CellHack2

Quick start Rules Reference About

Reference

Everything a cell can see and do, precisely. You do not need this page to start.

How your code is assembled

Your cell is a NASM source file in Intel syntax. The server prepends

```
bits 64
org CODE_BASE          ; 0x00100000
default rel
; every constant from cellhack.inc
```

and assembles it with `nasm -f bin`. Execution starts at your first byte. Do not write `%include` or `section` directives; you do not need them. The constants are listed in `cellhack.inc`.

Calling convention

System V AMD64, as for a C function:

Register	Meaning
`rdi`	pointer to the view (`VIEW_BASE`, read-only)
`rsi`	your energy, 0 to 50
`rdx`	pointer to your memory (`CELLMEM_BASE`, read/write)
`rsp`	16 KB of zeroed stack, with the return address on top
`al`	the action you return

All registers may be clobbered. Registers, flags and the stack are reset before every call; only the first `MEM_SIZE` (64) bytes at `[rdx]` persist. The rest of that 4 KB page is scratch, zeroed before each call. Your code

region is read-only; keep mutable data in `[rdx]` or on the stack. Only `al` is read as the action; the rest of `rax` is ignored.

The view (`[rdi]`, 128 bytes)

Offset	Name	Type	Meaning
0	`VIEW_TICK`	u32	current tick, from 0
4	`VIEW_ENERGY`	u8	your energy (same as `rsi`)
6	`VIEW_RAND`	u8	a random byte, fresh every call
7	`VIEW_AGE`	u8	ticks since you were born, capped at 255
8	`VIEW_X`	u16	your column, 0 at the left
10	`VIEW_Y`	u16	your row, 0 at the top
12	`VIEW_W`	u16	arena width
14	`VIEW_H`	u16	arena height
16	`VIEW_POP_SELF`	u16	living cells in your colony
18	`VIEW_POP_ENEMY`	u16	living cells in the other colonies
20	`VIEW_COLONY`	u8	your colony number, 0 to 3
32	`VIEW_CELLS`	u8[25]	the 5×5 neighbourhood: `CELL_EMPTY` 0, `CELL_FRIEND` 1, `CELL_ENEMY` 2, `CELL_WALL` 3
64	`VIEW_ENERGIES`	u8[25]	energy of the cell on each of those squares, 0 if none

Unused bytes are zero.

Neighbourhood indices

The 25 squares are row by row, top-left first, with you at index 12:

```
 0  1  2  3  4
 5  6  7  8  9
10 11 12 13 14
15 16 17 18 19
20 21 22 23 24
```

Index for an offset `(dx, dy)` with each in −2…2: `(dy + 2) * 5 + (dx + 2)`. The four squares next to you have names: `VIEW_UP` 7, `VIEW_RIGHT` 13, `VIEW_DOWN` 17, `VIEW_LEFT` 11. Cell types and energies are both bytes: `byte [rdi + VIEW_CELLS + VIEW_UP]` and `byte [rdi + VIEW_ENERGIES + VIEW_UP]`.

Actions (return in `al`)

Value	Name	Effect
`0x01`	`ACT_REST`	gain `REST_GAIN` (1)
`0x10 + dir`	`ACT_MOVE + dir`	step into that square
`0x20 + dir`	`ACT_SPLIT + dir`	new cell there with half your energy; needs `SPLIT_MIN` (5)
`0x30 + dir`	`ACT_EAT + dir`	take up to `EAT_TAKE` (2) from the cell there
`0x40 + dir`	`ACT_FEED + dir`	give `FEED_AMOUNT` (2) to the cell there
`0x00`	`ACT_NOP`	nothing

`dir` is `DIR_UP` 0, `DIR_RIGHT` 1, `DIR_DOWN` 2 or `DIR_LEFT` 3. `ACT_DIR_MASK` (3) masks a random byte down to a direction. Any other value is a `bad_action` fault.

Limits and faults

- **Instructions:** `INSN_BUDGET` (1000) per call; past it the call is a `budget` fault. The engine counts every instruction. The count decides initiative within the tick (fewest first) and breaks ties at the end of a match (fewest in total). The match page shows your colony's average and maximum per call.
- **Size:** source up to 64 KB, assembled code up to `CODE_MAX` (32 KB).
- **Not allowed:** `syscall`, `sysenter`, `int`, `int3`, `ud2`, `in`, `out`, `rdrand`. `rdtsc` works and returns the tick. The CPU is a plain `qemu64`: SSE2 and SSE3 are available; anything newer is not guaranteed.
- **Memory faults:** reading, writing or executing unmapped addresses, writing to the code or view pages, and running off the stack.

A fault means no action this tick. The first faults of each match are logged with tick, position, kind and `rip`, visible to the cell's owner on the match page.

Memory map

Region	Address	Size	Access
`CODE_BASE`	`0x00100000`	64 KB	read, execute; your code, padded with `int3`
`STACK_BASE`	`0x00200000`	16 KB	read, write; zeroed each call, guard pages either side
`VIEW_BASE`	`0x00300000`	4 KB	read; the view
`CELLMEM_BASE`	`0x00310000`	4 KB	read, write; first 64 bytes persist
`EXIT_ADDR`	`0x00400000`	4 KB	the return address; arriving here ends the call

Constants

Every name on this page is defined in `cellhack.inc`, which is prepended to your source automatically.